ЗВІТ

Практична робота №14

Тема: Версіонування схем і міграції

Мета роботи

Навчитися фіксувати зміни структури бази даних у вигляді послідовності міграцій, створювати та застосовувати міграції, документувати їх виконання і розглядати можливість відкату змін.

1. Початковий стан схеми — міграція V1

Для виконання роботи використано структуру бази даних college з практичної роботи №8.

Початковий стан містить десять таблиць:

1. AUTHORS — автори книг.
2. BOOK_AUTHORS — зв'язок між книгами та авторами.
3. BOOKS — інформація про книги.
4. COPIES — примірники книг.
5. COURSES — навчальні курси.
6. DEPARTMENTS — факультети.
7. ENROLLMENTS — записи студентів на курси.
8. LOANS — видача та повернення книг.
9. PUBLISHERS — видавництва.
10. READERS — читачі.

Початкова структура створюється міграцією V1__init.sql.

У ній визначаються таблиці, первинні та зовнішні ключі, обмеження цілісності й типи даних.

Результат: створено початкову структуру бази даних із десяти таблиць.

2. Додавання обов'язкового стовпця — міграція V2

У другій міграції V2__add_teacher_id.sql необхідно додати до таблиці COURSES обов'язковий стовпець teacher_id.

Для забезпечення посилальної цілісності створюється таблиця TEACHERS, яка зберігає інформацію про викладачів.

Оскільки таблиця COURSES може вже містити записи, не можна просто додати обов'язковий стовпець без значень для наявних рядків.

Тому використовується така послідовність:

1. Створюється таблиця TEACHERS.
2. Додається початковий запис викладача.
3. До COURSES додається стовпець teacher_id із значенням за замовчуванням.
4. Встановлюється обмеження NOT NULL.
5. Додається зовнішній ключ до TEACHERS.
6. Значення DEFAULT видаляється, щоб надалі викладача потрібно було вказувати явно.

Результат: кожен курс обов'язково пов'язаний із викладачем.

3. Зміна обмеження — міграція V3

У третій міграції V3__drop_status.sql змінюється обмеження кількості кредитів курсу.

Початкове обмеження дозволяло значення від 1 до 30. Після міграції дозволений діапазон становить від 1 до 60.

Для цього старе обмеження видаляється та створюється нове.

Зворотний крок полягає у видаленні нового обмеження та відновленні початкового діапазону від 1 до 30.

Важливо, що такий відкат можливий лише тоді, коли в таблиці немає курсів із кількістю кредитів понад 30.

Результат: змінено правило перевірки даних без видалення таблиці або її записів.

4. Журнал застосування міграцій

Для відстеження виконаних міграцій використовується журнал flyway_schema_history.

Він зберігає інформацію про версію міграції, її назву, час виконання та результат.

Очікувана послідовність виконання:

Версія	Назва	Результат
V1	init	Створено 10 початкових таблиць
V2	add_teacher_id	Додано таблицю TEACHERS та стовпець teacher_id
V3	drop_status	Змінено обмеження кількості кредитів

Якщо міграція вже була успішно виконана, Flyway розпізнає її за журналом і не застосовує повторно.

Це запобігає повторному створенню таблиць, стовпців та обмежень.

5. Зворотні міграції

Зворотна міграція дозволяє повернути структуру бази даних до попереднього стану.

Для виконаних змін передбачено такі зворотні дії:

- **V1:** видалення створених таблиць у зворотному порядку залежностей. Така операція призводить до втрати даних.
- **V2:** видалення зовнішнього ключа, стовпця teacher_id та таблиці TEACHERS.
- **V3:** відновлення початкового обмеження credits BETWEEN 1 AND 30.

Перед виконанням відкату необхідно враховувати залежності між таблицями та можливу втрату даних.

6. Приклад складної багатокрокової міграції

Розглянемо ситуацію, коли необхідно зробити стовпець teacher_id обов'язковим у таблиці COURSES, але таблиця вже містить записи.

Одразу встановити NOT NULL неможливо, якщо наявні записи не мають значень для цього стовпця.

Тому зміна виконується кількома етапами:

1. Створити таблицю викладачів.
2. Додати викладача для початкового заповнення.
3. Додати стовпець teacher_id, який спочатку допускає NULL.
4. Заповнити значення для всіх наявних курсів.
5. Додати зовнішній ключ.
6. Встановити обмеження NOT NULL.

Такий підхід дозволяє поступово змінити структуру бази даних без порушення цілісності наявних даних.

7. Контрольні запитання

1. Чому міграції зберігають у репозиторії разом із кодом?

Це дозволяє зберігати історію змін бази даних, контролювати версії схеми та відтворювати однакову структуру в різних середовищах розробки.

2. Що таке зворотна міграція і коли вона потрібна?

Зворотна міграція — це набір операцій, які повертають структуру бази даних до попереднього стану. Вона потрібна у випадку помилкової зміни або необхідності скасувати оновлення.

3. Як міграції допомагають узгодити схеми у різних командах?

Усі учасники використовують однакову послідовність версій міграцій. Це дозволяє підтримувати однакову структуру бази даних у різних середовищах та уникати розбіжностей.

4. Чому зміну NOT NULL на таблиці з даними складно зробити одним кроком?

Якщо в таблиці вже є записи з NULL у відповідному стовпці, встановлення NOT NULL порушить обмеження цілісності. Спочатку потрібно заповнити відсутні значення, перевірити їх і лише після цього встановити обмеження.

Висновок

Під час виконання практичної роботи було розглянуто принципи версіонування структури бази даних та застосування міграцій.

Було підготовлено три послідовні міграції: початкове створення схеми, додавання обов'язкового зовнішнього ключа для викладачів та зміну обмеження кількості кредитів.

Також розглянуто ведення журналу міграцій, пропуск уже виконаних версій, зворотні кроки та поетапне внесення складних змін до структури бази даних.

У результаті було опрацьовано підхід, який дозволяє контролювати зміни схеми та підтримувати узгодженість бази даних у різних середовищах.